

FICHE DE RÉVISION ULTIME — UNE VIE DE FOURMI

Romain • Lisa • Yannis

Tout ce qu'il faut savoir pour défendre le projet

Équipe : Romain • Lisa • Yannis

Pitch de 20 secondes — « On modélise la fourmilière comme un graphe. Comme plusieurs fourmis circulent simultanément avec des capacités de salles, un simple plus court chemin ne suffit pas. On déplie donc le graphe dans le temps et on cherche un flot de F unités. On teste T dans l'ordre croissant : le premier T faisable est le nombre minimal d'étapes. »

1. Les 6 règles du sujet

- Toutes les fourmis partent de S_v et doivent finir dans S_d .
- À chaque étape : attendre ou aller dans une salle voisine.
- Capacité d'une salle intermédiaire = 1 par défaut, ou X pour $SN\{X\}$.
- S_v et S_d sont non limitants.
- Les mouvements d'une même étape sont simultanés.
- Objectif : faire arriver l'intégralité des fourmis en un minimum d'étapes.

2. Vocabulaire minimum

Terme	Dans le projet	Phrase jury
Sommet	Une salle	« Une salle devient un sommet. »
Arête	Un tunnel	« Un tunnel relie deux sommets. »
Adjacence	Deux salles voisines	« Une arête signifie qu'un déplacement est possible. »
Capacité	Nb max de fourmis	« Elle évite la saturation d'une salle. »
Flot	Trajectoires agrégées	« Une unité de flot représente une fourmi. »
Horizon T	Nb d'étapes testé	« On cherche le plus petit T faisable. »

Piège numéro 1 : BFS / plus court chemin résout un trajet individuel. Le projet demande d'optimiser le trafic de F fourmis avec des capacités partagées.

3. L'algorithme — à savoir expliquer sans code

- Calculer d — d = longueur du plus court chemin $S_v \rightarrow S_d$. Avant d étapes, personne ne peut arriver.

- Tester $T = d$ — on demande si toutes les fourmis peuvent arriver en T étapes.
- Déplier le temps — on crée une copie de chaque salle aux temps $0, 1, \dots, T$.
- Encoder les actions — $salle(t) \rightarrow voisine(t+1) = \text{déplacement}$; $salle(t) \rightarrow \text{même salle}(t+1) = \text{attente}$.
- Encoder les capacités — chaque salle est scindée en entrée/sortie avec une arête de capacité X .
- Calculer un flot maximal — si la valeur du flot vaut F , les F trajectoires existent simultanément.
- Sinon augmenter T — on teste $T+1$, puis $T+2$, etc.
- Premier T faisable = minimum — on décompose ensuite le flot en $f_1, f_2, \dots$ pour afficher les étapes.

4. La preuve d'optimalité — phrase parfaite

« Pour un horizon T , le graphe temporel représente tous les déplacements autorisés pendant T étapes et respecte les capacités. Un flot de valeur F signifie donc qu'une solution complète existe en T étapes. Comme nous testons T dans l'ordre croissant à partir de d , le premier T faisable est nécessairement optimal. »

5. Pourquoi un min-cost-flow après ?

Le nombre d'étapes est déjà optimal. Le coût sert seulement à choisir une solution plus lisible : favoriser les arrivées tôt dans S_d et éviter les attentes ou détours inutiles.

À ne pas confondre : le min-cost-flow ne rend pas T optimal. L'optimalité vient de la recherche du premier horizon faisable.

6. Exemple du sujet — 3 fourmis

Étape	Mouvements
E1	$f_1 : S_v \rightarrow S_1$; $f_2 : S_v \rightarrow S_2$
E2	$f_1 : S_1 \rightarrow S_d$; $f_2 : S_2 \rightarrow S_d$; $f_3 : S_v \rightarrow S_1$
E3	$f_3 : S_1 \rightarrow S_d$

Point subtil : à E2, f_3 peut entrer dans S_1 pendant que f_1 en sort : les mouvements de l'étape sont simultanés.

7. Architecture du repository

Fichier	Rôle
ants.py	Modèle, parsing, graphe temporel, flot, validation
main.py	CLI, fichier/dossier, export des résultats
visualization.py	Graphe, images étape par étape, GIF
tests/test_ants.py	Tests automatiques
README.md	Problématique, solution, installation, preuve, sources

8. Commandes + fichiers produits

Installation :

```
python -m venv .env
# activer l'environnement
pip install -r requirements.txt
```

Un cas :

```
python main.py inputs/cas_simple.txt
```

Tous les cas :

```
python main.py inputs
```

Tests :

```
python -m unittest discover -s tests -v
```

Fichiers générés :

- solution.txt : étapes E1, E2, ...
- graphe.png : salles + tunnels.
- matrice_adjacence.csv : matrice 0/1.
- animation.gif : progression complète.
- etapes/etape_XX.png : état à chaque instant.

9. Fonctions importantes

Fonction	Rôle
parse_anthill_text	Lit F, les tunnels et les capacités.
Anthill.solve	Cherche le premier horizon faisable.
_build_time_expanded_graph	Construit les copies temporelles et les capacités.
nx.maximum_flow	Répond : « F fourmis peuvent-elles arriver en T étapes ? »
nx.min_cost_flow	Choisit une solution lisible à horizon optimal.
_decompose_flow	Transforme le flot en trajectoires f1, f2, ...
_validate_solution	Revérifie tunnels, arrivée et capacités.

10. Tests à citer

- Cas simple du sujet → 3 étapes.
- Tunnel direct Sv-Sd → 1 étape même pour plusieurs fourmis.
- Chemin avec salle de capacité 1 → effet de pipeline.
- Salle de capacité 2 → deux fourmis peuvent cohabiter.

- Graphe sans chemin → erreur claire.
- Validation finale : chaque mouvement est une arête et chaque capacité est respectée.

11. Questions pièges du jury — 1 à 9

1. Pourquoi un graphe non orienté ?

Les tunnels du sujet relient deux salles sans sens unique imposé.

2. Pourquoi pas seulement BFS ?

BFS optimise la distance d'un trajet individuel, pas le débit global de F fourmis.

3. Comment prouvez-vous l'optimalité ?

On teste T dans l'ordre croissant ; le premier horizon qui accepte un flot de valeur F est minimal.

4. Pourquoi découper une salle en entrée/sortie ?

Pour transformer une capacité de sommet en capacité d'arête, directement gérable par un algorithme de flot.

5. Les fourmis peuvent-elles attendre ?

Oui, grâce aux arêtes $salle(t) \rightarrow même\ salle(t+1)$.

6. Peut-on entrer pendant qu'une autre sort ?

Oui. La capacité est contrôlée au nouvel instant, ce qui autorise le remplacement simultané.

7. Les tunnels ont-ils une capacité ?

Le sujet n'en donne pas ; nous limitons donc les salles, conformément à l'énoncé.

8. Que représente une unité de flot ?

Une fourmi.

9. Pourquoi le flot est-il décomposable en fourmis ?

Les capacités sont entières ; le flot obtenu est entier et peut être suivi unité par unité.

12. Questions pièges du jury — 10 à 18

10. Pourquoi $d + F - 1$ est une borne haute ?

Sur un chemin simple de longueur d, même avec capacité 1, on peut injecter une nouvelle fourmi à chaque étape.

11. Différence max-flow / min-cost-flow ?

Max-flow teste la faisabilité ; min-cost choisit une solution plus lisible parmi les solutions à horizon optimal.

12. Pourquoi Edmonds-Karp ?

Il est simple, déterministe et largement suffisant pour les tailles pédagogiques visées.

13. Pourquoi exporter une matrice d'adjacence ?

C'est une représentation classique du graphe et une notion explicitement donnée dans la base de connaissances du sujet.

14. Que se passe-t-il si Sd est inaccessible ?

Le programme le détecte avant la résolution et renvoie une erreur claire.

15. Pourquoi séparer visualisation et algorithme ?

Pour tester la logique indépendamment et garder une architecture maintenable.

16. Quelle est la limite principale ?

Le réseau temporel grossit avec T, les salles et les tunnels.

17. Pourquoi pas une stratégie gloutonne ?

Un bon choix local peut bloquer une meilleure organisation globale ; elle ne garantit pas l'optimum.

18. Amélioration future ?

Interface graphique interactive, import automatique des jeux officiels et comparaison de plusieurs solutions optimales.

13. Les 7 erreurs à ne surtout pas dire

→ « On utilise Dijkstra » — le cœur est un flot temporel, pas un plus court chemin pondéré.

→ « Le plus court chemin donne la solution » — faux avec plusieurs fourmis et des capacités.

→ « Une salle doit être vide avant le début de l'étape » — incomplet : son occupante peut partir pendant la même étape.

→ « Min-cost-flow rend le nombre d'étapes optimal » — non : l'optimalité vient du premier horizon faisable.

→ « Un tunnel ne laisse passer qu'une fourmi » — ce n'est pas une règle du sujet.

→ « NetworkX fait tout » — la modélisation temporelle, le parseur, la décomposition, la validation et les sorties sont notre travail.

→ Lire du code pendant plusieurs minutes — expliquer l'architecture et le raisonnement est plus convaincant.

14. Complexité — réponse honnête

« Le réseau temporel contient environ $(T+1)$ copies de chaque salle, plus les arêtes de déplacement et d'attente à chaque étape. La taille augmente donc avec T, le nombre de salles et le nombre de tunnels. C'est un compromis assumé pour garantir l'optimalité dans le cadre du projet. »

15. Conclusion qui fait propre

« Nous avons choisi une modélisation plus ambitieuse qu'un simple calcul de chemin parce qu'elle correspond exactement au problème : plusieurs agents, des ressources limitées, des mouvements simultanés et un objectif de temps global minimal. »

16. Sources à connaître

- Sujet fourni : Une vie de fourmi — La Plateforme.
- NetworkX Tutorial : networkx.org/documentation/stable/tutorial.html
- Adjacency Matrix - Graph Theory Tutorial : people.revoledu.com/kardi/tutorial/GraphTheory/Adjacency-Matrix.html